Program - 8: To Study Continuous Integration Tool

Aim: To study and understand the working of a continuous integration (CI) tool (e.g., Jenkins / GitLab CI / GitHub Actions) for automating the building, testing, and integration of code in a software development environment.

Objectives

- To learn the concept of Continuous Integration and its importance in software development.
- To set up and configure a CI tool to automate builds & testing.
- To understand the pipeline stages – build, test, deploy.
- To analyze the results and logs generated by CI tool.

Problem Statement

Manual integration of code from multiple developers often lead to conflict, build failures, and delays in software delivery. A CI tool is needed to automatically integrate code changes, run tests, and generate build artifacts to improve software quality and development speed.

Description

Continuous Integration (CI) is a DevOps practice where code changes from all developers are automatically integrated into a shared repository several times a day. Each integration triggers an automated build and test process to detect integration issues early.

A CI tool can:

- Pull code automatically from version control (like GitHub).
- Build the code into executable or deployable packages.
- Run automated tests to ensure code quality.
- Provide feedback to developers immediately.

This experiment involves installing, accessing a CI tool, connecting it to a version control repository, configuring a build pipeline, and observing the automation of the build/test process.

Procedure

1. **Install / Access CI tool:** Install Jenkins or use a cloud-based CI tool like GitHub Actions.
2. **Connect to repository:** Link the CI tool to a version control system (GitHub / GitLab).
3. **Create a new project / pipeline:** In the CI dashboard create a new job or pipeline.
4. **Configure Build steps:**
    - Specify repository URL
    - Define build commands (compile / test)
5. **Set triggers:** Enable automatic builds on each code commit or schedule builds.

6. **Run the pipeline:** Commit a change to the repository to trigger pipeline.
7. **Monitor build and test results:** View logs, test reports, and artifacts.
8. **Analyze outcomes:** Check if the build succeeded and test passed, and troubleshoot if errors occur.

Steps

Step 1 – Clone the Repository

Clone the existing GitHub repository to your local computer:

- git clone <repo-link>

Step 2 – Move into the Repository

Open the cloned repository:

- cd repo_name

Step 3 – Create GitHub Actions Workflow Folder

Create the .github/workflows directory:

- mkdir -p .github/workflows

This folder is used to store GitHub Actions workflow files.

Step 4 – Create the Workflow File

Create a workflow file called deploy.yml:

- code .github/workflows/deploy.yml

The deploy.yml file contains the instructions that GitHub Actions will execute automatically.

Step 5 – Add Workflow Content

Add the required GitHub Actions code inside deploy.yml.
For example (Syntax):

```
name: CI workflow

on:
  push:
    branches:
      - master

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - name: check out code
        uses: actions/checkout@v4
```

```
    - name: Run CI
      run: echo "Build and Test Successful"
```

This workflow runs automatically whenever code is pushed to the master branch. *Note: If your repository uses main instead of master, change master to main.*

Step 6 – Add the Workflow File

Add the newly created file to Git:

- git add .github/workflows/deploy.yml

Step 7 – Commit the Changes

Commit the changes with a meaningful message:

- git commit -m "Add GitHub Actions workflows"

Step 8 – Push the Changes

Push the changes to GitHub:

- git push origin master

(If your branch is main, use: git push origin main)

Step 9 – Check GitHub Actions

1. Open your GitHub Repository in the browser.
2. Go to: **Repository -> Actions**

You will see the workflow. GitHub Actions will automatically start executing the workflow after the push.

Step 10 – Check Workflow Result

Click on the workflow to view its execution details. If everything is successful, you will see:

- CI workflow
- Build
- Run CI

The workflow will show a successful / green status. If there is an error, GitHub Actions will show a failed status and provide logs to identify the problem.

Conclusion

The experiment successfully demonstrated the use of a continuous integration tool to automate building and testing of software whenever code changes occur. CI improves the overall efficiency of software development.